

Aula 04 — Métodos Não Paramétricos (KNN)

Curso de Aprendizado de Máquina — UFRJ

Leitura recomendada: AME §4.1–4.5; ISLP §3.5 e cap. 7

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- explicar o que distingue um método *não paramétrico* de um paramétrico e por que ele troca viés por variância;
- descrever a estratégia de *bases* (séries ortogonais, *splines*) para flexibilizar a regressão linear;
- definir o estimador de k **vizinhos mais próximos** (KNN) e o papel de k no balanço viés–variância;
- definir o estimador de **Nadaraya–Watson** e reconhecer os principais *kernels* de suavização e o papel da janela h ;
- entender a **regressão polinomial local** como extensão de Nadaraya–Watson e sua vantagem na fronteira;
- enxergar todos esses métodos como *médias locais* / *suavizadores lineares*.

Esta aula é uma mudança de paradigma em relação às Aulas 01–02. Lá fixávamos a forma de r — linear, polinomial de grau p — e estimávamos poucos parâmetros. Von Neumann resumiu o desconforto dessa postura numa frase famosa: “*com quatro parâmetros eu ajusto um elefante; com cinco, faço a tromba mexer.*” A pergunta desta aula é a consequência natural:

e se eu não quiser supor forma alguma para $r(\mathbf{x})$?

A resposta é deixar os *dados* ditarem a forma. O preço, você já sabe qual é — e a Aula 05 vai mostrar que ele é bem mais alto do que parece quando d cresce.

1 O que é um método não paramétrico?

Um método **paramétrico** supõe que $r(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$ pertence a uma família indexada por um número *fixo e finito* de parâmetros (ex.: regressão linear, $d + 1$ coeficientes). Um método **não paramétrico** não impõe uma forma funcional fixa: a complexidade efetiva do modelo *crece com n* . A suposição é mais fraca — tipicamente apenas que r é *suave*.

Atenção: “não paramétrico” não quer dizer “sem parâmetros”

O nome engana. O KNN guarda a base de treino inteira — em certo sentido, tem *mais* parâmetros que uma regressão linear, não menos. O que caracteriza o método é que o número de graus de liberdade *não é fixado de antemão*: ele acompanha n . Com 50 pontos você tem um modelo; com 50 mil, outro, mais rico.

Ideia central

Em relação aos paramétricos, métodos não paramétricos **reduzem o viés** (capturam relações arbitrárias) ao custo de **aumentar a variância**. Continuamos no mundo da Aula 01: o segredo é controlar a flexibilidade por um *tuning parameter* (o k do KNN, a janela h do kernel), escolhido por validação cruzada (Aula 03).

Observação 1.1. Há uma ponte importante com a Aula 02. Penalizar um modelo muito flexível (Ridge, Lasso) e usar um modelo não paramétrico são duas formas de se posicionar no mesmo balanço viés-variância, vindas de lados opostos: a penalização *aproxima* um modelo flexível de um simples; o não paramétrico *parte* do flexível e só o segura o quanto for preciso.

2 Estratégia de bases: séries ortogonais e *splines*

A primeira ideia para ganhar flexibilidade sem sair do arcabouço linear é **ampliar o conjunto de atributos** com funções $\phi_1, \dots, \phi_I$ das covariáveis e ajustar

$$g(x) = \sum_{j=1}^I \beta_j \phi_j(x),$$

que é *linear* nos β_j — estimável por mínimos quadrados (Aula 02). Aqui a flexibilidade é controlada por I (número de funções da base).

2.1 Séries ortogonais

Usa-se uma base ortonormal de L^2 (Fourier, polinômios ortogonais, *wavelets*). Aproxima-se r por suas primeiras I componentes na base; I é o *tuning parameter* (mais termos $\Rightarrow$ mais flexível). É a ideia por trás de “regressão polinomial” vista na Aula 01 (a base dos monômios $1, x, x^2, \dots$).

2.2 *Splines*

Splines são funções polinomiais por partes, “coladas” suavemente em pontos chamados **nós** (*knots*) $t_1, \dots, t_p$. Uma base conveniente para *splines* de grau k é

$$f_1(x) = 1, f_2(x) = x, \dots, f_{k+1}(x) = x^k, \quad f_{k+1+j}(x) = (x - t_j)_+^k, \quad j = 1, \dots, p,$$

onde $(u)_+ = \max(u, 0)$. Com isso $I = k + 1 + p$, e novamente estimam-se os coeficientes por mínimos quadrados.

Ideia central: por que a base truncada funciona

Vale entender por que $(x - t_j)_+^k$ é a peça certa. Essa função é identicamente nula à esquerda de t_j e um polinômio de grau k à direita. Somá-la ao modelo, portanto, *não muda nada antes do nó* e permite mudar a curvatura *depois* dele — e como ela e suas $k - 1$ primeiras derivadas se anulam em t_j , a emenda sai suave de graça. Cada nó compra exatamente um grau de liberdade a mais, localizado onde você quiser.

Variantes importantes: *B-splines* (base numericamente estável, é o que as bibliotecas usam de fato), *natural splines* (impõem linearidade fora dos nós extremos, reduzindo a variância na fronteira — mesma preocupação da Seção 5) e *smoothing splines* (em vez de escolher nós, põem um nó em cada observação e penalizam a curvatura $\int (g''(x))^2 dx$ — caso particular de RKHS, §4.6 do [AME]).

Poucos nós dão uma curva rígida (subajuste); muitos nós dão uma curva nervosa (superajuste). O número e a posição dos nós são o botão de complexidade — é a Aula 01 de novo, com outro nome.

3 k vizinhos mais próximos (KNN)

O KNN é talvez o método não paramétrico mais intuitivo: para prever em $\mathbf{x}$, faça a *média das respostas dos k pontos de treino mais próximos de $\mathbf{x}$* .

Definição 3.1 (Regressão KNN). Seja $\mathcal{N}_{\mathbf{x}}$ o conjunto dos índices das k observações de treino mais próximas de $\mathbf{x}$ (segundo uma distância d , em geral euclidiana): $\mathcal{N}_{\mathbf{x}} = \{i : d(\mathbf{X}_i, \mathbf{x}) \leq d_{\mathbf{x}}^k\}$, onde $d_{\mathbf{x}}^k$ é a distância do k -ésimo vizinho. Então

$$\hat{r}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_{\mathbf{x}}} Y_i. \quad (1)$$

Repare no que essa fórmula *não* tem: nenhum parâmetro a estimar, nenhuma otimização, nenhum ajuste. O “treino” do KNN consiste em guardar os dados. Todo o trabalho acontece na hora da predição — por isso o método é chamado de *preguiçoso* (*lazy learner*).

O parâmetro k controla a flexibilidade:

- **k pequeno** (ex.: $k = 1$): vizinhança minúscula, segue cada ponto $\Rightarrow$ *viés baixo, variância alta* (curva “nervosa”, superajuste).
- **k grande**: média sobre muitos pontos; quando $k \rightarrow n$, $\hat{r}$ tende à média global (constante) $\Rightarrow$ *viés alto, variância baixa* (subajuste).

Escolhe-se k por validação cruzada.

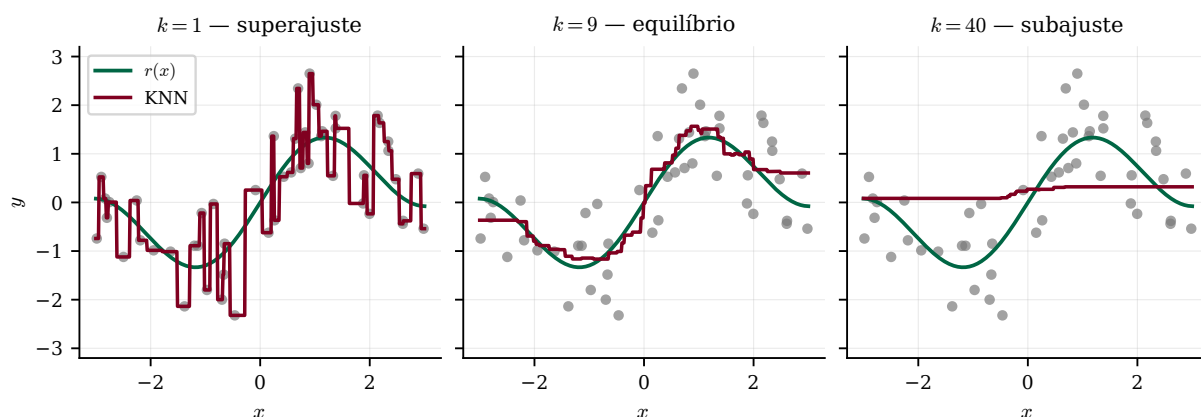


Figura 1: KNN sobre a mesma amostra de $n = 50$ pontos da Aula 01. Com $k = 1$ a estimativa passa por cima de cada observação e vira uma escada — é a versão KNN do polinômio de grau 15. Com $k = 40$, quase toda a amostra entra em cada média e a curva quase não distingue um x do outro. O k faz exatamente o papel que o grau do polinômio fazia, só que *ao contrário*: aqui é o k *pequeno* que significa flexível.

Atenção

KNN exige uma noção de distância e, portanto, **depende fortemente da escala** das covariáveis. Se uma variável está em reais e outra em milhares de reais, a segunda domina completamente a distância euclidiana e a primeira poderia nem existir. *Padronize* antes (Aula 07). Além disso, o KNN sofre da *maldição da dimensionalidade*: em d alto, “vizinhos” deixam de ser próximos — é o tema inteiro da Aula 05.

4 Nadaraya–Watson (regressão por kernel)

O KNN dá peso $1/k$ aos k vizinhos e 0 aos demais — uma transição abrupta. Nadaraya–Watson suaviza isso: usa uma **média ponderada** de *todas* as observações, com pesos que decaem com a distância.

$$\hat{r}(\mathbf{x}) = \sum_{i=1}^n w_i(\mathbf{x}) Y_i, \quad w_i(\mathbf{x}) = \frac{K(\mathbf{x}, \mathbf{X}_i)}{\sum_{j=1}^n K(\mathbf{x}, \mathbf{X}_j)}, \quad (2)$$

onde K é um **kernel de suavização** que mede a similaridade entre $\mathbf{x}$ e $\mathbf{X}_i$. Note $\sum_i w_i(\mathbf{x}) = 1$. Escolhas comuns (com janela/largura $h > 0$):

Kernel	$K(\mathbf{x}, \mathbf{X}_i)$
Uniforme	$\mathbb{I}\{d(\mathbf{x}, \mathbf{X}_i) \leq h\}$
Gaussiano	$(2\pi h^2)^{-1/2} \exp(-d^2(\mathbf{x}, \mathbf{X}_i)/(2h^2))$
Triangular	$(1 - d(\mathbf{x}, \mathbf{X}_i)/h) \mathbb{I}\{d(\mathbf{x}, \mathbf{X}_i) \leq h\}$
Epanechnikov	$(1 - d^2(\mathbf{x}, \mathbf{X}_i)/h^2) \mathbb{I}\{d(\mathbf{x}, \mathbf{X}_i) \leq h\}$

O kernel uniforme dá peso igual a todos dentro do raio h (e zero fora); os demais ponderam pela proximidade; o gaussiano dá peso positivo a todos. A janela h é o *tuning parameter*: h grande $\Rightarrow$ muita suavização (viés alto, variância baixa); h pequeno $\Rightarrow$ pouca (variância alta, viés baixo).

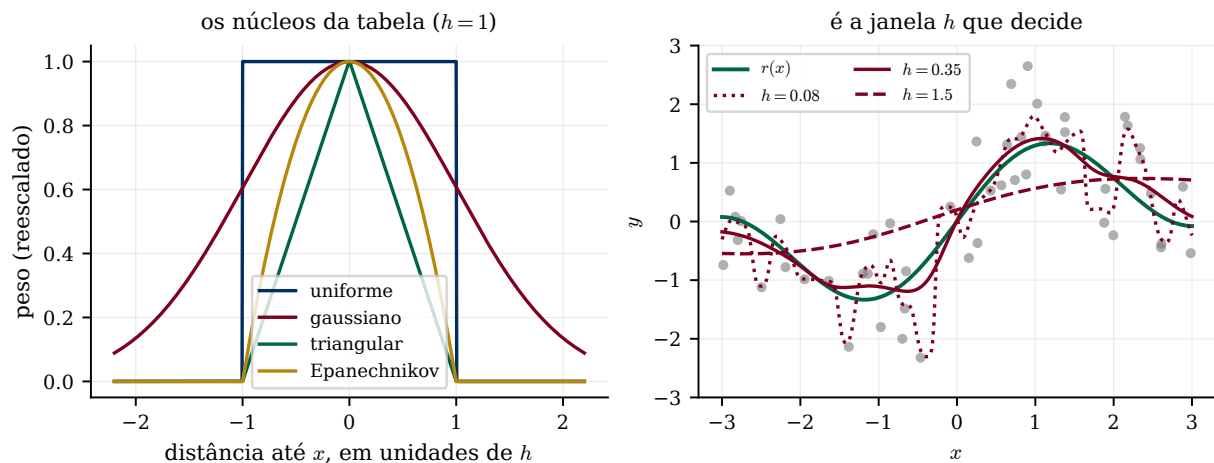


Figura 2: À esquerda, os quatro núcleos da tabela, todos com $h = 1$ e reescalados para topo 1: eles diferem no formato, mas todos fazem a mesma coisa — dar mais peso a quem está perto. À direita, o que *de fato* muda o ajuste: com o mesmo núcleo gaussiano, $h = 0,08$ persegue o ruído, $h = 1,5$ achata a senoide até quase uma reta, e $h = 0,35$ acerta.

Observação 4.1 (Na prática, a janela importa mais que o kernel). Empiricamente, a escolha do *kernel* pouco afeta o resultado; o que importa é o *tuning parameter* h (ou k). A Figura 2 explica por quê: os quatro núcleos são versões ligeiramente diferentes do mesmo perfil, enquanto mudar h altera a *escala* da vizinhança em ordens de grandeza. Foque a validação cruzada em h e não perca tempo comparando núcleos.

4.1 Nadaraya–Watson como mínimos quadrados ponderados locais

Há uma leitura iluminadora: $\hat{r}(\mathbf{x})$ em (2) é o intercepto $\hat{\beta}_0$ que resolve

$$\hat{\beta}_0 = \arg \min_{\beta_0} \sum_{i=1}^n w_i(\mathbf{x}) (Y_i - \beta_0)^2. \quad (3)$$

Ou seja: em cada ponto $\mathbf{x}$, ajustamos uma **constante** por mínimos quadrados, ponderando as observações pela proximidade a $\mathbf{x}$. É um ajuste linear *local*.

Demonstração. Derivando (3) em β_0 e igualando a zero: $-2 \sum_i w_i(\mathbf{x})(Y_i - \beta_0) = 0 \Rightarrow \beta_0 = \frac{\sum_i w_i(\mathbf{x})Y_i}{\sum_i w_i(\mathbf{x})} = \sum_i w_i(\mathbf{x})Y_i$, pois os pesos somam 1. Logo $\hat{\beta}_0$ coincide com (2). $\square$

Ideia central

Essa releitura é mais do que uma curiosidade: ela abre a porta da próxima seção. Se ajustar uma *constante* localmente já dá um método útil, nada impede ajustar uma *reta*, ou uma *parábola*. É a mesma equação com mais termos.

5 Regressão polinomial local

A regressão polinomial local (LOESS/LOWESS) substitui o intercepto por um polinômio de grau p :

$$\hat{\beta}_0, \dots, \hat{\beta}_p = \arg \min_{\beta_0, \dots, \beta_p} \sum_{i=1}^n w_i(\mathbf{x}) \left(Y_i - \beta_0 - \sum_{j=1}^p \beta_j x_i^j \right)^2, \quad (4)$$

e a predição em $\mathbf{x}$ é $\hat{\beta}_0$. Em forma matricial, com $\mathbf{B}$ tendo i -ésima linha $(1, x_i, \dots, x_i^p)$ e $\mathbf{\Omega} = \text{diag}(w_i(\mathbf{x}))$:

$$(\hat{\beta}_0, \dots, \hat{\beta}_p)^\top = (\mathbf{B}^\top \mathbf{\Omega} \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{\Omega} \mathbf{Y}.$$

Para *cada* $\mathbf{x}$ resolve-se um problema diferente (os pesos mudam). Tomando $p = 0$ recupera-se Nadaraya–Watson.

5.1 O problema da fronteira

A vantagem do grau $p \geq 1$ aparece nas *bordas* do domínio, e vale entender o mecanismo. Tome $\mathbf{x}$ na ponta direita do suporte. A vizinhança de $\mathbf{x}$ só tem pontos *de um lado* — todos à esquerda. Se r está subindo ali, a média local desses pontos é uma média de valores *menores* que $r(\mathbf{x})$, e a estimativa fica sistematicamente puxada para baixo. Não é ruído: é viés, e ele não desaparece por mais dados que você colete.

A reta local não cai nessa armadilha porque ela *estima a inclinação* e extrapola com ela. A Figura 3 mede o efeito.

Atenção: o grau 1 não é de graça

Na mesma simulação, o desvio-padrão na fronteira *sobe* de 0,108 (grau 0) para 0,138 (grau 1): estimar uma inclinação a partir de poucos pontos de um lado só é uma tarefa ruidosa. Você trocou viés por variância outra vez — e se n for pequeno, a troca pode não compensar. É a Aula 01, agora dentro de uma janela.

6 Uma visão unificadora: suavizadores lineares

KNN, Nadaraya–Watson, *splines* e regressão local são todos **suavizadores lineares**: a predição é uma combinação linear das respostas,

$$\hat{r}(\mathbf{x}) = \sum_{i=1}^n \ell_i(\mathbf{x}) Y_i,$$

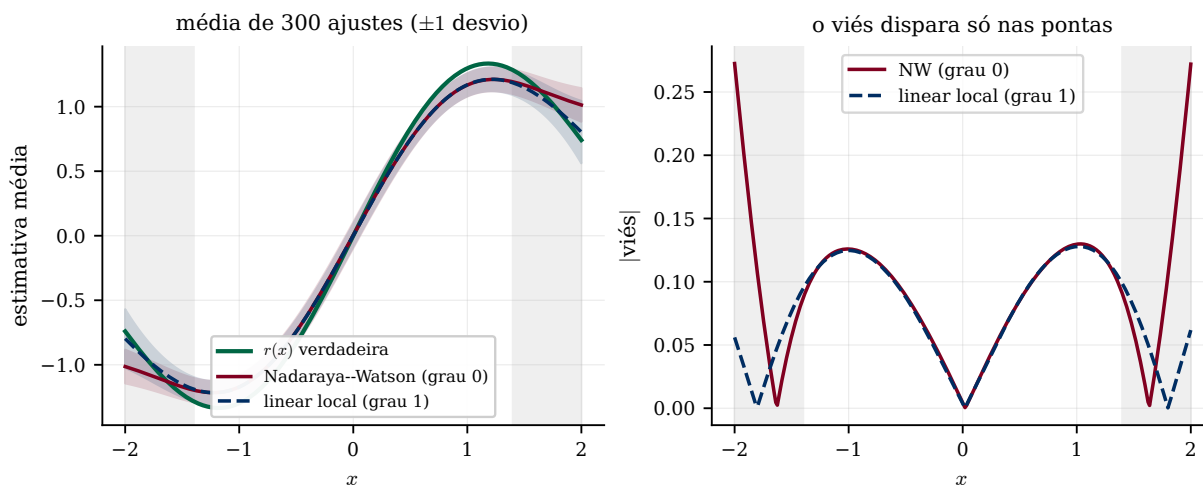


Figura 3: Viés medido em 300 amostras independentes ($n = 200$, $h = 0,35$, núcleo gaussiano). À esquerda, a estimativa *média* de cada método com uma faixa de ± 1 desvio-padrão: no miolo as duas acompanham r , mas nas pontas a curva do Nadaraya–Watson descola. À direita, o mesmo em números: o viés é idêntico nos dois métodos ao longo de todo o interior e *dispara* apenas nas faixas cinza. Em média sobre a fronteira, o viés cai de 0,097 (grau 0) para 0,045 (grau 1) — menos da metade.

com pesos $\ell_i(\mathbf{x})$ que dependem só das covariáveis (não de $\mathbf{Y}$). Para o KNN, $\ell_i(\mathbf{x}) = \frac{1}{k} \mathbb{I}\{i \in \mathcal{N}_{\mathbf{x}}\}$; para NW, $\ell_i(\mathbf{x}) = w_i(\mathbf{x})$.

Ideia central

Vista assim, a diferença entre os métodos desta aula é só *o formato dos pesos*: o KNN usa uma janela retangular de largura variável (larga onde há poucos dados, estreita onde há muitos); o Nadaraya–Watson, uma janela de largura fixa e bordas suaves. Essa estrutura comum explica por que todos enfrentam o mesmo dilema — largura da vizinhança $\leftrightarrow$ complexidade — e por que valem para todos os atalhos vistos antes, como o do LOOCV via *leverage* (Aula 03, Prop. 3.1): basta que $\ell_i(\mathbf{x})$ não dependa de $\mathbf{Y}$.

7 Prática em Python

```

1 from sklearn.neighbors import KNeighborsRegressor
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4 from sklearn.model_selection import GridSearchCV
5
6 # KNN com k escolhido por validacao cruzada (padronizando dentro do pipeline!)
7 modelo = make_pipeline(StandardScaler(), KNeighborsRegressor())
8 grade = {"kneighborsregressor_n_neighbors": [1, 3, 5, 11, 25, 51]}
9 busca = GridSearchCV(modelo, grade, cv=5, scoring="neg_mean_squared_error")
10 busca.fit(X_tr, y_tr)
11 print("melhor k:", busca.best_params_)

```

O `StandardScaler` dentro do *pipeline* não é preciosismo: sem ele o KNN mede distâncias em unidades misturadas, e com ele a padronização é refeita dentro de cada dobra, sem vazamento (Aulas 03 e 07).

Para Nadaraya–Watson e regressão local, `statsmodels` traz `lowess` e `KernelReg` (em `statsmodels.nonparametric`). O notebook Aula prática 03, 04 e 05 explora k e h em dados artificiais e no `superconductivity.csv`.

Resumo

- Métodos não paramétricos não fixam a forma de r ; reduzem viés e aumentam variância em relação aos paramétricos. A complexidade cresce com n .
- *Bases* (séries, *splines*): linearizam o problema; flexibilidade via número de termos/nós.
- **KNN (1)**: média dos k vizinhos; k controla viés-variância, com k *pequeno* = flexível (Fig. 1).
- **Nadaraya–Watson (2)**: média ponderada por kernel; a janela h é o que importa, não o kernel (Fig. 2); é uma constante ajustada localmente (3).
- **Regressão polinomial local**: polinômio local; corta o viés de fronteira pela metade, ao custo de mais variância lá (Fig. 3).
- Todos são *suavizadores lineares*; *padronize* as covariáveis e cuidado com a dimensão (Aula 05).

Leitura recomendada. [AME] §4.1–4.2 (séries e *splines*), §4.3 (KNN, com a Figura 4.2 sobre o efeito de k), §4.4 (Nadaraya–Watson, Figuras 4.3 e 4.4 — esta última compara dois kernels e mostra o quanto isso importa pouco) e §4.5 (regressão polinomial local). [ISLP] §3.5 (regressão linear $\times$ KNN, onde fica claro quando o paramétrico ainda ganha) e Capítulo 7 inteiro (*Moving Beyond Linearity*), em especial §7.4–7.6 sobre *splines* e regressão local.

Para praticar. [recursos/listas/Lista de exercícios 04.pdf](#).